



Universidad Autónoma Metropolitana
Unidad Cuajimalpa

Protocolo de Proyecto Terminal I

**Plataforma distribuida de comunicación y alertas comunitarias
para entornos universitarios (Noti Uam)**

Alumno: Aaron Rodrigo Ramos Reyes
Carrera: Sistemas y Tecnologías de Información
Trimestre: 26-P
Asesor: Dr. Guillermo Monroy Rodriguez
Universidad: UAM Cuajimalpa

Ciudad de México, México
2026

Plataforma distribuida de comunicación y alertas comunitarias para entornos universitarios (Noti Uam)

Aaron Rodrigo Ramos Reyes
Universidad Autónoma Metropolitana, Unidad Cuajimalpa
Ciudad de México, México
aaron.r.amos@cua.uam.mx

RESUMEN

La comunicación dentro de las universidades suele estar fragmentada entre múltiples plataformas externas (Foros, redes sociales, correo institucional), lo que provoca baja visibilidad de eventos académicos, desinformación y dificultad para descubrir canales de interés. Este proyecto terminal presenta **Noti Uam**, una plataforma distribuida de comunicación y alertas comunitarias orientada a entornos universitarios. El sistema se basa en una arquitectura distribuida *publish-subscribe* (pub-sub) que permite la creación de canales temáticos, suscripción abierta y notificaciones en tiempo real. Se propone implementar el backend con Spring Boot y Apache Kafka, utilizar PostgreSQL y Redis para persistencia y caché, y desarrollar una aplicación móvil multiplataforma con Flutter. El objetivo general es diseñar y construir una solución escalable, usable y accesible que mejore la difusión de información académica, cultural y comunitaria dentro de la UAM Cuajimalpa. El presente documento detalla el planteamiento del problema, los fundamentos teóricos, el estado del arte, los objetivos, la hipótesis, la metodología incremental y el cronograma de trabajo para los tres trimestres de Proyecto Terminal.

CCS CONCEPTS

• **Human-centered computing** → **Collaborative and social computing systems and tools**; • **Security and privacy** → *Management and access control*.

KEYWORDS

notificaciones académicas, sistemas distribuidos, publish-subscribe, Apache Kafka, Flutter, usabilidad

1. INTRODUCCIÓN

1.1. Contexto de la comunicación en entornos universitarios

La comunicación efectiva dentro de las instituciones de educación superior constituye un pilar fundamental para el desarrollo de actividades académicas, culturales y sociales. Diariamente, estudiantes, profesores y personal administrativo generan y consumen una gran cantidad de información: avisos de clase, cambios de horario, fechas de examen, convocatorias a eventos, formación de grupos de estudio, difusión de talleres, entre muchos otros mensajes. Sin embargo, la forma en que esta información se distribuye actualmente en la mayoría de las universidades —incluyendo la Universidad Autónoma Metropolitana (UAM)— dista de ser óptima.

Diversos estudios han documentado que los estudiantes recurren de manera masiva a plataformas externas como *WhatsApp*, *Facebook*, *Instagram*, *Telegram* o el correo electrónico institucional para

comunicarse y mantenerse informados [1]. Estas herramientas, si bien son de fácil acceso y uso cotidiano, no fueron diseñadas para gestionar la comunicación estructurada y escalable de una comunidad universitaria. Como señala Bucheli (2020), el uso de aplicaciones de mensajería instantánea en el ámbito académico se ha dado de manera espontánea y no regulada, lo que genera tanto oportunidades como problemas de fragmentación y desinformación.

1.2. Problemática identificada

La dependencia de plataformas no especializadas produce múltiples efectos negativos:

- **Fragmentación de la información:** Los anuncios importantes suelen dispersarse en decenas de grupos de WhatsApp, páginas de Facebook o cadenas de correo. Un estudiante que no pertenece a un grupo específico puede perderse avisos relevantes sobre su curso o su carrera.
- **Dificultad para descubrir contenido:** En plataformas como WhatsApp, los grupos son cerrados y requieren una invitación. Esto impide que los alumnos conozcan la existencia de comunidades de estudio, eventos culturales o asesorías abiertas [1].
- **Riesgo de desinformación:** Al no existir mecanismos de verificación o moderación, cualquier usuario —incluso ajeno a la institución— puede publicar contenido falso o irrelevante. Esto afecta la credibilidad de la comunicación institucional.
- **Baja efectividad del correo institucional:** Aunque formal, el correo electrónico suele ser revisado con poca frecuencia por los estudiantes, y los sistemas de notificaciones asociados son lentos o poco visibles.

Bucheli (2020) encontró, en un estudio realizado en la Universidad Autónoma del Estado de Hidalgo, que más del 80 % de los estudiantes utilizan WhatsApp para comunicarse con sus compañeros y profesores, pero al mismo tiempo reportan sentir “sobrecarga de información” y “estrés por la mezcla de temas personales y académicos”. Esta situación se replica en muchas universidades públicas mexicanas, incluida la UAM Cuajimalpa.

1.3. Insuficiencia de las soluciones existentes

Algunas universidades han implementado sistemas propios de gestión de aprendizaje (como Moodle, Blackboard o plataformas personalizadas). Estos sistemas, si bien son útiles para entregar tareas y consultar calificaciones, no están diseñados para la difusión ágil de notificaciones en tiempo real ni para la creación de canales comunitarios abiertos. Por otra parte, aplicaciones como *Telegram* ofrecen canales públicos y notificaciones, pero su adopción no es homogénea en toda la comunidad universitaria y siguen siendo herramientas externas sin integración con los servicios institucionales.

En el ámbito de los sistemas distribuidos y las notificaciones en tiempo real, existen arquitecturas probadas como el modelo *publish-subscribe* (pub-sub) que permite desacoplar a los productores de información de los consumidores, garantizando escalabilidad y entrega eficiente de eventos [10, 11]. Sin embargo, muy pocas plataformas universitarias han adoptado este tipo de arquitecturas para la comunicación cotidiana.

1.4. Propuesta de solución: Noti Uam

Ante este panorama, se propone el desarrollo de **Noti Uam**, una plataforma distribuida de comunicación y alertas comunitarias específicamente orientada a entornos universitarios. La plataforma se basa en una arquitectura *publish-subscriber* (pub-sub) que permite:

- La creación de **canales temáticos** (por asignatura, por actividad deportiva, por área de interés) visibles para toda la comunidad.
- La **suscripción voluntaria y abierta** a dichos canales, sin necesidad de invitaciones previas.
- El envío de **notificaciones en tiempo real** a los dispositivos móviles de los usuarios suscritos.
- Un sistema de autenticación seguro mediante JWT, que garantice que solo miembros de la comunidad universitaria puedan publicar (evitando spam o desinformación externa).

La plataforma será desarrollada como una **aplicación móvil multiplataforma** usando *Flutter*, dado que los dispositivos móviles son el principal medio de acceso a internet entre los estudiantes. El *backend* se implementará con *Spring Boot* y *Apache Kafka* para la gestión de eventos, mientras que el almacenamiento de datos se realizará con *PostgreSQL* y *Redis* para optimizar el rendimiento. Todo el sistema se contenerizará con *Docker* para facilitar su despliegue y escalabilidad.

1.5. Contribución del trabajo

La principal contribución de este proyecto es proporcionar una solución tecnológica adaptada a las necesidades específicas de comunicación de la UAM Cuajimalpa, que:

- Reduce la fragmentación informativa al centralizar los avisos en una sola aplicación.
- Favorece el descubrimiento abierto de actividades académicas y culturales.
- Ofrece notificaciones inmediatas basadas en una arquitectura distribuida robusta (pub-sub).
- Incorpora principios de Interacción Humano-Computadora (IHC) para garantizar usabilidad y accesibilidad.

Además, desde el punto de vista académico, el proyecto permite aplicar y demostrar competencias en sistemas distribuidos, bases de datos, desarrollo móvil y metodologías de diseño centrado en el usuario.

1.6. Estructura del protocolo

El presente protocolo se organiza de la siguiente manera: después de esta introducción, el **Marco Teórico** (sección 2) expone los conceptos fundamentales de sistemas distribuidos, el modelo pub-sub, las tecnologías seleccionadas y los principios de IHC. El **Estado del Arte** (sección 3) revisa soluciones similares existentes

y las compara. La sección 4 detalla los **objetivos** general y específicos. La sección 5 plantea la **hipótesis** de trabajo. La **Metodología** (sección 6) describe el enfoque incremental y las fases de desarrollo. Finalmente, la sección 7 presenta el **cronograma** tentativo y la sección 8 lista las referencias bibliográficas.

2. MARCO TEÓRICO

El presente marco teórico expone los conceptos fundamentales necesarios para comprender y justificar las decisiones tecnológicas y de diseño de la plataforma **Noti Uam**. Se abordan, en primer lugar, los principios de los sistemas distribuidos y, en particular, el modelo de arquitectura *publisher-subscriber* (pub-sub). Posteriormente, se describen las tecnologías específicas seleccionadas para el desarrollo del *backend*, la base de datos, la capa de mensajería y la aplicación móvil. Finalmente, se introducen los principios de Interacción Humano-Computadora (IHC) y los mecanismos de autenticación y seguridad.

2.1. Sistemas distribuidos: principios y paradigmas

Un **sistema distribuido** es aquel en el que componentes de software y hardware ubicados en equipos de red conectados se comunican y coordinan mediante el paso de mensajes, con el objetivo de compartir recursos y ofrecer servicios de manera unificada a los usuarios [4]. Tanenbaum y Van Steen (2023) definen un sistema distribuido como “una colección de computadoras independientes que aparece ante sus usuarios como un solo sistema coherente”.

Las propiedades deseables de los sistemas distribuidos incluyen:

- **Transparencia:** ocultar al usuario y al programador la complejidad de la distribución (ubicación, concurrencia, fallos, etc.).
- **Escalabilidad:** capacidad de crecer en número de nodos o usuarios sin degradar el rendimiento.
- **Tolerancia a fallos:** continuar operando, incluso si algunos componentes fallan.
- **Desacoplamiento en tiempo y espacio:** los productores y consumidores de información no necesitan estar activos simultáneamente ni conocerse entre sí.

La última propiedad es particularmente relevante para sistemas de notificaciones como *Noti Uam*, donde los emisores de avisos (estudiantes, profesores) y los receptores (suscriptores) operan de manera asíncrona y pueden estar desconectados en distintos momentos.

2.2. Arquitectura publisher-subscriber (pub-sub)

El modelo **publisher-subscriber** es un estilo arquitectónico para sistemas de comunicación basada en eventos [11]. En este modelo, los productores (*publishers*) generan eventos o mensajes y los publican en un *broker* o intermediario sin conocer a los suscriptores. Los consumidores (*subscribers*) expresan su interés en ciertos tipos de eventos (normalmente mediante filtros o canales) y el *broker* se encarga de entregar los mensajes a todos los suscriptores relevantes.

Las ventajas principales del modelo pub-sub son:

- **Desacoplamiento espacial:** publicadores y suscriptores no necesitan saber la dirección o identidad de los demás.

- **Desacoplamiento temporal:** no es necesario que ambas partes estén activas al mismo tiempo; el *broker* almacena y reenvía los mensajes cuando el suscriptor esté disponible.
- **Escalabilidad:** se pueden añadir nuevos nodos publicadores o suscriptores sin modificar la lógica central.

Estas características hacen de pub-sub una arquitectura ideal para sistemas de alertas y notificaciones en tiempo real, como el propuesto en este proyecto. Según Tarkoma (2012), las implementaciones modernas de pub-sub pueden manejar millones de eventos por segundo con latencias del orden de milisegundos.

2.3. Componentes tecnológicos del backend

2.3.1. Apache Kafka. **Apache Kafka** es una plataforma distribuida de *streaming* de eventos diseñada originalmente por LinkedIn. Se ha convertido en un estándar de facto para construir tuberías de datos en tiempo real y aplicaciones reactivas [7]. Kafka implementa un modelo de *log distribuido* que combina las ventajas de pub-sub con almacenamiento duradero y alta capacidad de *throughput*.

En *Noti Uam*, Kafka actuará como el *broker* central de eventos. Los productores (API REST del backend) publicarán mensajes en *topics* (equivalentes a canales temáticos) y los consumidores (servicio de notificaciones push) se suscribirán a esos *topics* para enviar alertas a los dispositivos móviles. La naturaleza distribuida de Kafka permite escalar horizontalmente para atender a toda la comunidad universitaria.

2.3.2. Spring Boot. **Spring Boot** es un marco de trabajo de Java que simplifica la creación de aplicaciones basadas en el ecosistema Spring, proporcionando configuración automática y servidores embebidos (como Tomcat) [9]. Se utiliza ampliamente en la industria para desarrollar microservicios y *backends* RESTful.

En la plataforma propuesta, Spring Boot será el encargado de:

- Exponer los endpoints de la API REST para gestionar usuarios, canales y publicaciones.
- Integrarse con Kafka mediante la plantilla *KafkaTemplate* para publicar eventos.
- Implementar la lógica de autenticación y autorización mediante JWT.
- Servir de puente entre la aplicación móvil y los servicios internos.

2.3.3. PostgreSQL y Redis. El almacenamiento de datos persistentes se realizará con **PostgreSQL**, un sistema de gestión de bases de datos relacional y de código abierto, conocido por su fiabilidad, concurrencia y cumplimiento de ACID [5]. Se utilizará para almacenar usuarios, canales, suscripciones y publicaciones históricas.

Por otro lado, **Redis** es un almacén de estructura de datos en memoria que actúa como base de datos, caché y *message broker* ligero. En *Noti Uam*, Redis se empleará para:

- Cachear consultas frecuentes (por ejemplo, los canales más populares) y reducir la carga sobre PostgreSQL.
- Gestionar información temporal, como sesiones activas y tokens de notificaciones push.
- Almacenar contadores de eventos no leídos para cada usuario.

La combinación de PostgreSQL (persistencia) y Redis (rendimiento) es común en sistemas distribuidos con requisitos de baja latencia [10].

2.3.4. Contenerización con Docker. **Docker** permite empaquetar aplicaciones y sus dependencias en contenedores ligeros y portátiles. Cada servicio (Spring Boot, Kafka, PostgreSQL, Redis) se ejecutará en su propio contenedor, definiendo un archivo *docker-compose.yml* que orqueste la red y el orden de inicio. Esto facilita el despliegue tanto en entornos de desarrollo como en producción, garantizando la reproducibilidad del sistema.

2.4. Aplicación móvil multiplataforma: Flutter

Flutter es un kit de desarrollo de interfaz de usuario creado por Google que permite compilar aplicaciones nativas para iOS, Android, Web y escritorio desde una sola base de código en el lenguaje Dart [8]. Su arquitectura basada en *widgets* reactivos y su alto rendimiento (sin puente JavaScript) lo hacen adecuado para aplicaciones que requieren animaciones fluidas y notificaciones en tiempo real.

En *Noti Uam*, Flutter permitirá:

- Desarrollar simultáneamente las versiones para Android e iOS, llegando a la mayoría de los estudiantes.
- Integrar notificaciones push mediante *Firebase Cloud Messaging* (FCM) o *OneSignal*.
- Consumir la API REST del backend mediante el paquete *http dio*.
- Implementar una interfaz moderna, accesible y centrada en la usabilidad, siguiendo los principios de IHC.

2.5. Interacción Humano-Computadora (IHC)

La Interacción Humano-Computadora es una disciplina que estudia el diseño, evaluación e implementación de sistemas interactivos para el uso humano [3]. Uno de sus objetivos centrales es mejorar la **usabilidad**, definida por la norma ISO 9241-11 como “el grado en que un producto puede ser utilizado por usuarios específicos para alcanzar metas específicas con efectividad, eficiencia y satisfacción en un contexto de uso”.

Para *Noti Uam*, los principios de IHC guiarán:

- El diseño de la navegación: permitir a los usuarios descubrir canales fácilmente, suscribirse y publicar avisos en pocos pasos.
- La accesibilidad: cumplir con pautas como contraste suficiente, tamaños de texto ajustables y compatibilidad con lectores de pantalla (accesibilidad móvil).
- La retroalimentación: notificaciones visuales y hápticas confirmatorias de acciones.
- La prevención de errores: validación de formularios, mensajes claros y deshacer acciones.

Se realizarán pruebas de usabilidad con estudiantes reales durante el desarrollo, aplicando métodos como el *System Usability Scale* (SUS) o pruebas de tareas con captura de tiempos.

2.6. Seguridad y autenticación: JWT

La plataforma debe garantizar que solo los miembros autenticados de la comunidad universitaria puedan publicar contenido

(aunque cualquier persona pueda leer los canales públicos si así se configura). Para ello, se implementará autenticación basada en tokens JWT (*JSON Web Tokens*) [6].

El flujo típico es:

1. El usuario inicia sesión con credenciales institucionales (correo UAM y contraseña, o integración con SSO de la universidad).
2. El backend valida las credenciales y genera un token JWT firmado que contiene el identificador del usuario y sus roles.
3. El cliente (app móvil) envía el token en el encabezado *Authorization* de cada petición.
4. El backend verifica la firma y los permisos antes de procesar la solicitud.

Los tokens JWT son autónomos (no requieren almacenamiento en servidor), lo que facilita la escalabilidad horizontal. Su tiempo de expiración será corto (por ejemplo, 1 hora) y se complementará con un *refresh token* para mejorar la experiencia de usuario.

2.7. Síntesis del marco teórico aplicado a Noti Uam

La combinación de los conceptos expuestos permite diseñar *Noti Uam* como un sistema distribuido, desacoplado, escalable y seguro, orientado a la alta usabilidad. La siguiente tabla resume la correspondencia entre cada capa tecnológica y los principios teóricos:

Cuadro 1: Correspondencia entre componentes y fundamentos teóricos

Componente	Fundamento teórico
Apache Kafka	Modelo publisher-subscriber, desacoplamiento temporal/espacial
Spring Boot	Arquitectura RESTful, contenedor de servicios
PostgreSQL + Redis	Persistencia ACID + caché en memoria
Flutter	Desarrollo multiplataforma, IHC, notificaciones push
JWT	Autenticación stateless, seguridad
Docker	Contenerización, portabilidad, entornos reproducibles

Este marco teórico sustenta las decisiones técnicas que se detallan en la metodología y justifica la viabilidad del proyecto desde una perspectiva académica y profesional.

3. ESTADO DEL ARTE

3.1. Introducción

El Estado del Arte tiene como objetivo revisar y analizar las soluciones existentes —tanto generales como específicas— que abordan problemáticas similares a las que motivan el desarrollo de **Noti Uam**. Este análisis permite identificar las fortalezas y limitaciones de cada enfoque, así como detectar áreas de oportunidad que justifiquen una nueva propuesta. Al final de la sección se presenta una tabla comparativa que resume las características clave de las soluciones revisadas, y se extraen conclusiones que orientan el diseño de la plataforma propuesta.

3.2. Soluciones generales de mensajería y notificaciones

3.2.1. WhatsApp. WhatsApp es la aplicación de mensajería instantánea más utilizada en el mundo, y también la más adoptada de manera informal en entornos universitarios [1]. Su funcionamiento se basa en grupos cerrados (con límite de participantes y necesidad de invitación) y en la difusión de mensajes uno a uno o en listas de difusión. Si bien es eficaz para la comunicación rápida entre grupos pequeños, presenta limitaciones graves para el ámbito universitario:

- **Falta de descubrimiento:** Los estudiantes no pueden encontrar grupos temáticos (por ejemplo, “Taller de programación” o “Eventos culturales”) a menos que un miembro los invite.
- **Mezcla de contextos:** Los chats personales y académicos se entremezclan, generando sobrecarga de información y distracción.
- **Inexistencia de arquitectura distribuida orientada a eventos:** WhatsApp no ofrece canales temáticos persistentes ni un modelo de suscripción abierta, y sus notificaciones no son personalizables por tema dentro de la universidad.

Por estas razones, a pesar de su popularidad, WhatsApp no satisface los requisitos de una plataforma de comunicación universitaria abierta, escalable y enfocada en la difusión institucional.

3.2.2. Telegram. Telegram ofrece una aproximación más cercana a lo que se necesita. Introduce el concepto de **canales** (unidireccionales, para difusión) y **supergrupos** (con hasta 200 mil miembros). Los canales pueden ser públicos y descubribles mediante un enlace o búsqueda. Además, cuenta con un sistema de bots que podría extender funcionalidades.

Sin embargo, Telegram sigue siendo una plataforma externa a la institución, no integrada con los servicios de autenticación universitarios. Cualquier persona puede crear canales y difundir información, lo que abre la puerta a desinformación o spam. Tampoco ofrece una arquitectura de publicación-suscripción (pub-sub) nativa para desarrolladores, ni mecanismos de notificaciones basados en eventos académicos estructurados (como cambios de horario o fechas de examen). Por tanto, aunque es superior a WhatsApp en cuanto a apertura, no está diseñada específicamente para el contexto universitario.

3.2.3. Slack y Discord. Slack y Discord son plataformas de colaboración por equipos que organizan la comunicación en *workspaces*, canales y mensajes directos. Slack se orienta a entornos profesionales y Discord a comunidades de juegos, aunque ambas se utilizan cada vez más en educación. Permiten la integración con servicios externos mediante webhooks y bots.

Sus desventajas para el proyecto son:

- Requieren que un administrador cree el espacio y los canales, lo cual no es ágil para una universidad completa.
- La búsqueda y descubrimiento de canales públicos está limitada al espacio de trabajo.
- No están diseñadas para la difusión masiva de notificaciones push con alta prioridad (aunque lo soportan).
- La autenticación y el control de acceso requieren configuraciones adicionales para vincularse con cuentas institucionales.

A pesar de que Slack y Discord son herramientas robustas, su adopción en una universidad implicaría una migración forzada de toda la comunidad, a una plataforma no institucional. Además, no resuelven el problema de la fragmentación entre múltiples plataformas.

3.3. Soluciones específicas para entornos universitarios

3.3.1. Plataformas de gestión de aprendizaje (LMS). Los sistemas como Moodle, Blackboard o Ulicua ofrecen módulos de notificaciones y foros. Sin embargo, su enfoque principal es la gestión de cursos, tareas y calificaciones, no la comunicación comunitaria abierta. Las notificaciones suelen ser por correo electrónico o mediante paneles internos, y rara vez utilizan una arquitectura pub-sub en tiempo real. Además, suelen ser pesadas y poco ágiles para la difusión de eventos culturales o actividades extracurriculares. Como señala Bucheli (2020), los estudiantes no revisan con frecuencia los foros de los LMS, prefiriendo herramientas más inmediatas.

3.3.2. Sistemas de notificaciones push en proyectos académicos. En la literatura técnica se pueden encontrar diversos proyectos y prototipos que exploran el uso de notificaciones push en contextos educativos o de servicios. Por ejemplo, el análisis de la tecnología push para notificar eventos y mensajería grupal ha demostrado que la combinación de servicios como Firebase Cloud Messaging con una arquitectura cliente-servidor es efectiva para este tipo de aplicaciones. Sin embargo, estos trabajos suelen centrarse en casos de uso muy específicos (transporte escolar, eventos aislados) y no en una plataforma de comunicación comunitaria abierta con canales temáticos y arquitectura pub-sub distribuida. Tampoco incorporan principios de Interacción Humano-Computadora (IHC) de manera explícita [3].

3.3.3. Plataformas universitarias existentes: el caso de UAM App. La Universidad Autónoma Metropolitana ha puesto a disposición de su comunidad una aplicación móvil institucional denominada **UAM App**, cuyas funcionalidades se describen en el sitio oficial <https://www.uam.mx/appuam>. De acuerdo con esta documentación, la aplicación permite a los profesores consultar horarios, cursos impartidos, Unidades de Enseñanza-Aprendizaje (UEA) y el calendario de evaluaciones. Para los alumnos, ofrece acceso a datos escolares, horarios, historial académico, UEA programadas e historial de pagos. Adicionalmente, muestra información general como el calendario escolar y diversos reglamentos universitarios.

Si bien UAM App cumple una función valiosa como canal de consulta administrativa y académica, su diseño no aborda la problemática de la comunicación comunitaria en tiempo real. Se trata de una herramienta predominantemente informativa y unidireccional, que no permite:

- La creación de canales temáticos por parte de los propios estudiantes o profesores.
- Un sistema de suscripción abierta a comunidades de interés (grupos de estudio, eventos culturales, torneos deportivos).
- La publicación y difusión de avisos en tiempo real mediante una arquitectura de eventos.

Adicionalmente, se ha observado que la aplicación ya no se encuentra disponible en las tiendas oficiales de aplicaciones (Play

Store), lo que limita su acceso y actualización. Por tanto, UAM App no satisface la necesidad identificada en este proyecto, lo que refuerza la pertinencia de desarrollar una plataforma específica como **Noti Uam**.

3.4. Tabla comparativa

A continuación se presenta una tabla que compara las soluciones analizadas según criterios relevantes para el proyecto: apertura de canales, modelo de notificaciones, arquitectura subyacente, usabilidad reportada y capacidad de integración institucional.

Cuadro 2: Comparación de soluciones existentes

Solución	Canales abiertos	Notificaciones en tiempo real	Arquitectura distribuida	Integración institucional
WhatsApp	No (solo grupos cerrados)	Sí (push)	Centralizada (servidores de Meta)	No
Telegram	Sí (canales públicos)	Sí (push)	Centralizada (servidores de Telegram)	Parcial (bots)
Slack/Discord	Parcial (dentro del workspace)	Sí	Centralizada	No
Moodle	No (foros internos)	No (correo o consulta)	Monolítica	Sí (LDAP, SSO)
Apps institucionales	No (avisos unidireccionales)	Parcial (pull o push básico)	Variable	Sí
Noti Uam (propuesta)	Sí (canales temáticos públicos)	Sí (pub-sub con Kafka)	Distribuida (microservicios)	Sí (JWT, SSO planeado)

3.5. Crítica y conclusiones del Estado del Arte

Del análisis anterior se extraen las siguientes conclusiones:

1. **Ninguna de las soluciones generales** (WhatsApp, Telegram, Slack) está diseñada específicamente para el contexto universitario, y todas dependen de servidores externos fuera del control de la institución, lo que plantea riesgos de privacidad.
2. **Las plataformas LMS y las apps institucionales** actuales no ofrecen un modelo de comunicación comunitaria abierta, con canales creados por los usuarios y descubrimiento autónomo. Su arquitectura generalmente no está basada en pub-sub, lo que limita la inmediatez de las notificaciones.
3. **Existe un vacío evidente** que justifica el desarrollo de *Noti Uam*: una plataforma propia de la universidad, basada en estándares abiertos y tecnologías modernas (Kafka, Spring Boot, Flutter), que ofrezca tanto la inmediatez de las notificaciones push como la apertura y organización de los canales temáticos, todo ello con un fuerte énfasis en la usabilidad y la accesibilidad [3].

Por tanto, el proyecto **Noti Uam** no solo es pertinente, sino que constituye una contribución original y necesaria para mejorar la comunicación en la UAM Cuajimalpa, al tiempo que sirve como caso de estudio aplicado de sistemas distribuidos e interacción humano-computadora.

4. OBJETIVOS

4.1. Objetivo general

Desarrollar una plataforma distribuida de difusión universitaria basada en una arquitectura *publish-subscribe* (pub-sub) que permita a estudiantes, profesores y coordinación académica publicar y recibir notificaciones en tiempo real mediante canales de suscripción orientados a la alta usabilidad y al descubrimiento abierto de información dentro de la comunidad universitaria.

La plataforma será desarrollada como una aplicación móvil, debido a que los dispositivos móviles representan actualmente el principal medio de acceso a herramientas digitales y comunicación entre estudiantes.

4.2. Objetivos específicos

A partir del objetivo general, se definen los siguientes objetivos específicos, cada uno asociado a un conjunto de actividades técnicas y a las tecnologías seleccionadas en el marco teórico [10, 11]:

- 1. **Diseñar una arquitectura distribuida** basada en el modelo *publish-subscribe* para la gestión escalable de eventos y notificaciones en tiempo real, utilizando *Spring Boot* para el desarrollo del *backend*, *Apache Kafka* para la distribución de eventos y *Docker* para la contenerización y despliegue de servicios [7, 9].
- 2. **Implementar un sistema de canales y suscripciones** que permita la difusión y descubrimiento de información académica, cultural y comunitaria, utilizando *PostgreSQL* para el almacenamiento de datos, *Redis* para optimización y manejo de información temporal [5], y *JWT* para autenticación segura de usuarios [6].
- 3. **Desarrollar una aplicación móvil multiplataforma** utilizando *Flutter* para la interfaz móvil y el envío de notificaciones push, priorizando principios de usabilidad, accesibilidad y rapidez de interacción orientados al entorno universitario [3, 8].

4.3. Alcance y entregables

Al finalizar el proyecto, se obtendrán los siguientes entregables:

- Un *backend* funcional desplegable mediante contenedores Docker, que exponga una API REST para la gestión de usuarios, canales, publicaciones y suscripciones.
- Un sistema de eventos basado en *Apache Kafka* que gestione las notificaciones en tiempo real.
- Una base de datos *PostgreSQL* con el modelo relacional para almacenamiento persistente, complementada con *Redis* para caché.
- Una aplicación móvil desarrollada en *Flutter* para Android e iOS que permita:
 - Registro e inicio de sesión mediante autenticación JWT.
 - Creación y administración de canales temáticos.
 - Suscripción a canales públicos y recepción de notificaciones push.
 - Publicación de avisos dentro de los canales.
 - Búsqueda y descubrimiento de canales por categoría o palabra clave.

- Documentación técnica del sistema (diagramas de arquitectura, manual de instalación, guía de usuario).
- Reporte final del proyecto que incluya resultados de pruebas de usabilidad y rendimiento.

4.4. Relación con el marco teórico

Cada objetivo específico se fundamenta en los conceptos expuestos en el marco teórico. La siguiente tabla muestra dicha correspondencia:

Cuadro 3: Correspondencia entre objetivos específicos y fundamentos teóricos

Objetivo específico	Fundamento teórico
1. Arquitectura distribuida pub-sub	Modelo publish-subscribe [11], sistemas distribuidos [10], Spring Boot [9], Kafka [7]
2. Canales, suscripciones y autenticación	Persistencia con PostgreSQL [5], caché con Redis, autenticación JWT [6]
3. Aplicación móvil y usabilidad	Flutter [8], principios de Interacción Humano-Computadora [3]

5. HIPÓTESIS

5.1. Planteamiento de la hipótesis general

Con base en el problema identificado —la fragmentación de la comunicación universitaria debido al uso de plataformas externas no especializadas— y en los fundamentos teóricos de los sistemas distribuidos y la arquitectura *publish-subscribe* [10, 11], se formula la siguiente hipótesis:

“El desarrollo e implementación de una plataforma distribuida de comunicación basada en una arquitectura publish-subscribe (Noti Uam) reducirá significativamente los tiempos de difusión de información académica y comunitaria, al tiempo que aumentará la tasa de descubrimiento y suscripción a canales de interés por parte de los estudiantes, en comparación con el uso combinado de WhatsApp y correo institucional.”

6. METODOLOGÍA

6.1. Enfoque metodológico

El desarrollo de **Noti Uam** se llevará a cabo mediante una **metodología incremental**, que permite construir el sistema por partes funcionales, validar cada componente y reducir riesgos. Se ha escogido este enfoque para adaptarse a los criterios de entrega trimestrales, y así poder cumplir con los hitos y plazos establecidos por la CLTSI (Consejo de Licenciatura en Tecnologías de Sistemas de Información). A diferencia de un modelo en cascada rígido, la metodología incremental facilita la retroalimentación temprana y la corrección de errores sin afectar todo el sistema.

Cada incremento corresponde a una fase de desarrollo, y al final de cada una se obtiene un entregable parcial pero funcional. Las fases se iteran de manera secuencial, aunque algunas actividades pueden solaparse (por ejemplo, la documentación acompaña a todas las fases).

6.2. Fases de desarrollo detalladas

A continuación se describen las nueve fases que componen la metodología. Para cada fase se indican: objetivo, actividades, entregables, herramientas, criterios de aceptación y duración estimada (en semanas laborales, dentro del cronograma de 33 semanas).

6.2.1. Fase 1: Análisis de requisitos.

- **Objetivo:** Identificar y documentar los requisitos funcionales y no funcionales de la plataforma, validándolos con potenciales usuarios (estudiantes, profesores).
- **Actividades:**
 1. Realizar entrevistas a 5 estudiantes y 2 profesores de la UAM Cuajimalpa.
 2. Elaborar un cuestionario en línea (Google Forms) para recoger necesidades de comunicación.
 3. Definir casos de uso (diagrama UML) y priorizarlos.
 4. Especificar requisitos no funcionales: latencia máxima (2 segundos), disponibilidad (99,9 %), escalabilidad (1000 eventos/segundo), seguridad (autenticación JWT).
- **Entregables:** Documento de requisitos (versión 1.0), diagrama de casos de uso.
- **Herramientas:** PlantUML, Google Forms, procesador de textos (Preferencialmente LaTeX).
- **Criterios de aceptación:** Todos los requisitos funcionales deben estar priorizados y aprobados por el asesor; los no funcionales deben ser medibles.
- **Duración:** 2 semanas.

6.2.2. Fase 2: Diseño arquitectónico y modelo de datos.

- **Objetivo:** Definir la arquitectura distribuida (basada en microservicios y pub-sub) y el esquema de la base de datos.
- **Actividades:**
 1. Diseñar el diagrama de componentes (véase Figura 2).
 2. Especificar las API REST (Swagger/OpenAPI).
 3. Modelar el diagrama entidad-relación para PostgreSQL (tablas: usuarios, canales, suscripciones, publicaciones).
 4. Diseñar la estructura de datos en Redis (sesiones, caché de canales populares).
- **Entregables:** Diagrama de arquitectura, especificación OpenAPI, modelo relacional (SQL DDL), plan de índices.
- **Herramientas:** PlantUML, Swagger.
- **Criterios de aceptación:** El modelo debe cumplir con 3ª forma normal; la arquitectura debe ser revisada y aprobada por el asesor.
- **Duración:** 3 semanas.

6.2.3. Fase 3: Configuración del entorno de desarrollo y despliegue.

- **Objetivo:** Preparar un entorno reproducible con Docker para todos los servicios.
- **Actividades:**
 1. Crear Dockerfiles para Spring Boot, Kafka, PostgreSQL, Redis.
 2. Escribir docker-compose.yml para orquestar los contenedores.
 3. Configurar volúmenes para persistencia de datos.
 4. Establecer red interna entre servicios.

- **Entregables:** Repositorio Git con docker-compose.yml y Dockerfiles; script de inicio.
- **Herramientas:** Docker, Docker Compose, Git/GitHub.
- **Criterios de aceptación:** Ejecutar docker-compose up debe levantar todos los servicios sin errores y comunicarse entre sí.
- **Duración:** 2 semanas.

6.2.4. Fase 4: Desarrollo del backend (Spring Boot + Kafka).

- **Objetivo:** Implementar los microservicios de autenticación, gestión de canales y publicación de eventos.
- **Actividades:**
 1. Crear proyecto Spring Boot con módulos: security (JWT), usuario, canal, publicación.
 2. Configurar productor de Kafka para eventos de nueva publicación.
 3. Implementar API REST para crear canales, suscribirse, publicar avisos.
 4. Integrar JWT para proteger endpoints.
- **Entregables:** Código fuente del backend (Java), pruebas unitarias (JUnit), colección de Postman para pruebas de API.
- **Herramientas:** IntelliJ IDEA/Visual Studio Code, Maven, Spring Boot, Kafka Client.
- **Criterios de aceptación:** Todos los endpoints deben responder correctamente y las pruebas unitarias cubrir al menos el 70 % de las clases críticas.
- **Duración:** 6 semanas.

6.2.5. Fase 5: Implementación de la base de datos y caché.

- **Objetivo:** Crear el esquema PostgreSQL y configurar Redis para mejorar el rendimiento.
- **Actividades:**
 1. Ejecutar scripts DDL en PostgreSQL (tablas, secuencias, índices).
 2. Configurar pool de conexiones HikariCP en Spring Boot.
 3. Implementar caché de canales más consultados con Redis (TTL 5 minutos).
 4. Sincronizar contadores de publicaciones no leídas en Redis.
- **Entregables:** Scripts SQL, configuración de Redis en Spring Boot (CacheManager), pruebas de rendimiento con 100 consultas concurrentes.
- **Herramientas:** pgAdmin, Redis Insight, JMeter (para pruebas iniciales).
- **Criterios de aceptación:** Las consultas de listado de canales deben ejecutarse de manera responsiva, acorde a los tiempos de respuesta definidos.
- **Duración:** 3 semanas.

6.2.6. Fase 6: Desarrollo de la aplicación móvil (Flutter).

- **Objetivo:** Construir la interfaz de usuario multiplataforma (Android/iOS) que consuma la API y reciba notificaciones push.
- **Actividades:**
 1. Configurar proyecto Flutter con arquitectura MVC (o BLoC).
 2. Implementar pantalla de inicio de sesión (almacenamiento seguro de JWT).

3. Desarrollar pantalla de exploración de canales (lista, búsqueda).
 4. Implementar pantalla de detalle de canal y publicaciones.
 5. Integrar notificaciones push con Firebase Cloud Messaging (FCM).
- **Entregables:** Código fuente Flutter, archivo `google-services.json` (Android), `GoogleService-Info.plist` (iOS), APK de prueba.
 - **Herramientas:** Android Studio, VS Code, Flutter SDK, paquete `http`, `firebase_messaging`.
 - **Criterios de aceptación:** La aplicación debe ejecutarse en emuladores de Android 11+ y iOS 14+; las notificaciones push deben llegar en menos de 3 segundos.
 - **Duración:** 7 semanas.

6.2.7. Fase 7: Integración y pruebas funcionales.

- **Objetivo:** Verificar que todos los componentes trabajen juntos correctamente.
- **Actividades:**
 1. Desplegar el sistema completo con Docker en un servidor de pruebas.
 2. Ejecutar pruebas de integración con Postman (colección automatizada).
 3. Realizar pruebas de extremo a extremo (E2E) con un usuario simulado que crea un canal, publica y recibe notificación.
 4. Corregir errores detectados.
- **Entregables:** Reporte de pruebas de integración, reporte de errores, log de errores corregidos.
- **Herramientas:** Postman, Newman (línea de comandos), Docker Compose.
- **Criterios de aceptación:** El 100 % de los casos de prueba definidos en la fase 1 (vea sección 6.2.1) deben ser cubiertos de manera exitosa.
- **Duración:** 3 semanas.

6.2.8. Fase 8: Pruebas de usabilidad y rendimiento.

- **Objetivo:** Evaluar la experiencia de usuario y la capacidad de carga del sistema.
- **Actividades:**
 1. Medir latencia de Kafka y tiempo de entrega de notificaciones push.
 2. Aplicar el cuestionario SUS a 15 estudiantes (prueba de usabilidad).
 3. Observar a 5 usuarios mientras realizan tareas típicas (crear canal, suscribirse, publicar) y registrar tiempos y errores.
- **Entregables:** Gráficas de rendimiento, puntuación SUS promedio, informe de observaciones de usabilidad.
- **Herramientas:** Apache JMeter, Google Forms (SUS), cronómetro.
- **Criterios de aceptación:** La latencia media debe ser <2 segundos; puntuación SUS >70.
- **Duración:** 3 semanas.

6.2.9. Fase 9: Documentación, análisis final y presentación.

- **Objetivo:** Consolidar todos los resultados y preparar los materiales para la presentación del proyecto terminal.

■ **Actividades:**

1. Redactar el informe final.
 2. Elaborar manual de instalación y guía de usuario.
 3. Preparar presentación en diapositivas (PowerPoint / LaTeX).
 4. Ensayar la defensa con el asesor.
- **Entregables:** Protocolos I, II, III (PDF), código fuente en repositorio, manuales, presentación.
 - **Herramientas:** Overleaf / LaTeX, GitHub, Canva / PowerPoint.
 - **Criterios de aceptación:** Aprobación del asesor.
 - **Duración:** 4 semanas.

6.3. Diagramas de la metodología y la arquitectura

A continuación se presentan los diagramas clave que ilustran tanto el flujo metodológico como la solución técnica. Los diagramas fueron elaborados con PlantUML, Lucid.app entre otras herramientas de creación y modelado de diagramas.

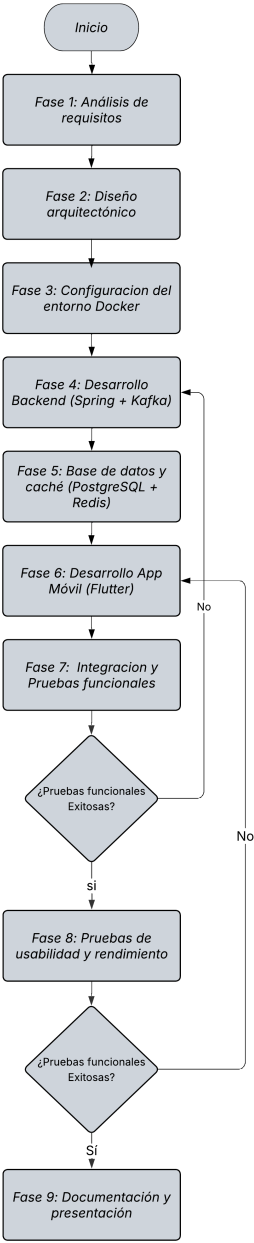


Figura 1: Flujo metodológico de las fases incrementales. Las flechas de retroalimentación indican ciclos de mejora.

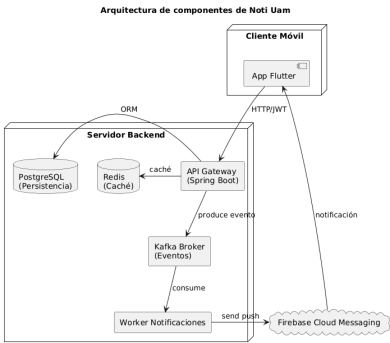


Figura 2: Arquitectura de componentes de Noti Uam. El API Gateway recibe peticiones, publica eventos en Kafka; el Worker consume y envía notificaciones push.

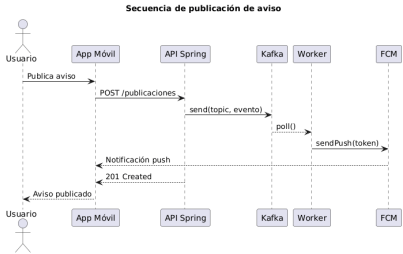


Figura 3: Secuencia de publicación de un aviso y entrega de notificación push a suscriptores.

Actividades para pruebas de usabilidad



Figura 4: Actividades para las pruebas de usabilidad con estudiantes.

6.4. Gestión de riesgos y contingencias

La siguiente tabla identifica los principales riesgos técnicos y organizacionales, junto con sus planes de mitigación.

Cuadro 4: Riesgos y planes de mitigación

Riesgo	Impacto	Mitigación
Fallo del broker Kafka	Pérdida de eventos en tiempo real	Configurar replicación de particiones y monitoreo con Prometheus; respaldo con Redis Streams como alternativa.
Alta latencia en notificaciones push	Mala experiencia de usuario	Optimizar la integración con FCM; usar colas de prioridad en Kafka.
Baja adopción en pruebas de usabilidad	Muestra insuficiente para validar hipótesis	Ofrecer incentivos; coordinar con profesores para solicitud de apoyo a la comunidad.
Problemas de compatibilidad entre Flutter y versiones de iOS/Android	Fallos en dispositivos reales	Probar en múltiples emuladores y dispositivos prestados; usar pruebas en la nube (Firebase Test Lab).
Sobrecarga de trabajo en el trimestre	Retraso en entregas	Priorizar funcionalidades críticas (MVP); aplicar metodología incremental para entregar al menos un prototipo funcional.

6.5. Herramientas de desarrollo y colaboración

- **Control de versiones:** Git + GitHub (repositorio privado).
- **Gestión de tareas:** GitHub Projects o Trello, con etiquetas por fase.
- **CI/CD:** GitHub Actions para ejecutar pruebas unitarias automáticamente en cada *push*.
- **Comunicación:** Discord / Slack para comunicación con el asesor.
- **Documentación:** Overleaf (LaTeX) para el informe y manuales.
- **Diagramas:** PlantUML (online) exportado a PNG.

6.6. Criterios de aceptación final del proyecto

Para dar por concluido el proyecto terminal, el sistema deberá cumplir con la siguiente lista de verificación:

1. Todos los objetivos específicos se han alcanzado según lo documentado en las fases.
2. La hipótesis general ha sido validada (o refutada, pero con conclusiones claras).

3. El código fuente está disponible en un repositorio Git con un README que explique cómo ejecutar el sistema.
4. El sistema se puede desplegar con `docker-compose up` en una máquina limpia.
5. La puntuación SUS media es superior a 70.
6. La documentación final (Protocolos I, II, III) ha sido aprobada por el asesor y otras autoridades pertinentes.

La metodología descrita garantiza un desarrollo sistemático, trazable y orientado a la validación de la hipótesis, lo que es esencial para el éxito del proyecto terminal.

7. CRONOGRAMA TENTATIVO

El proyecto se desarrolla en tres trimestres (Proyecto Terminal I, II y III), cada uno de 11 semanas. El diagrama siguiente muestra la distribución de actividades técnicas (basadas en las fases de la metodología) y los hitos establecidos por la CLTSI, adaptado al ancho de columna.

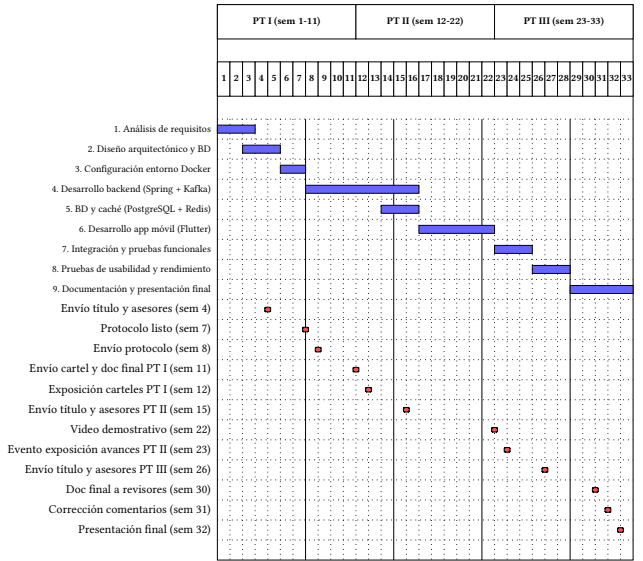


Figura 5: Cronograma detallado por semanas (33 semanas = 3 trimestres de 11 semanas). Las barras azules son las fases de la metodología; los hitos rojos son entregas obligatorias según la CLTSI.

7.1. Interpretación del cronograma

La distribución de actividades es la siguiente:

- **Trimestre I (semanas 1–11):** Se cubren las fases 1 a 4 (inicio del backend). Se entrega el protocolo en la semana 8 y el cartel en la 11.
- **Trimestre II (semanas 12–22):** Se completa el backend (fase 4), se implementa la base de datos y caché (fase 5), y se desarrolla la aplicación móvil (fase 6). El video demostrativo se entrega en la semana 22.
- **Trimestre III (semanas 23–33):** Se realizan las pruebas de integración, usabilidad y rendimiento (fases 7 y 8), y se

documenta el proyecto (fase 9). La presentación final ocurre en la semana 32.

Las fases 4 y 5 se solapan parcialmente (semanas 14–16) porque la base de datos y el backend se desarrollan en paralelo. Asimismo, la documentación (fase 9) comienza antes de que terminen las pruebas, para aprovechar el tiempo y permitir correcciones de última hora.

El cronograma es tentativo y podrá ajustarse durante el desarrollo, pero proporciona una guía clara para el cumplimiento de los hitos académicos.

REFERENCIAS

- [1] María Guadalupe Bucheli. 2020. *WhatsApp como herramienta de comunicación en estudiantes universitarios: caso de la Universidad Autónoma del Estado de Hidalgo*. Reporte técnico. Universidad Autónoma del Estado de Hidalgo.
- [2] Josiah L. Carlson. 2013. *Redis in Action*. Manning Publications, Shelter Island, NY.
- [3] Luis A. Castro and Marcela D. Rodríguez (Eds.). 2020. *Interacción Humano-Computadora y Aplicaciones en México* (2nd ed.). Academia Mexicana de Computación, A. C., México. <https://amexcomp.mx/media/publicaciones/int-humano-comp-apps.pdf>
- [4] George Coulouris, Jean Dollimore, and Tim Kindberg. 2011. *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley, Boston, MA.
- [5] PostgreSQL Global Development Group. 2024. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>. Documentación oficial de PostgreSQL.
- [6] M. Jones, J. Bradley, and N. Sakimura. 2015. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF).
- [7] Jay Kreps, Neha Narkhede, and Jun Rao. 2017. *Apache Kafka: A Distributed Streaming Platform*. <https://kafka.apache.org/documentation/>. Documentación oficial de Apache Kafka.
- [8] Google LLC. 2024. Flutter Documentation. <https://docs.flutter.dev/>. Documentación oficial de Flutter.
- [9] Pivotal Software. 2023. Spring Boot Reference Guide. <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Documentación oficial de Spring Boot.
- [10] Andrew S. Tanenbaum and Maarten Van Steen. 2023. *Distributed Systems: Principles and Paradigms* (4th ed.). Prentice Hall, Upper Saddle River, NJ.
- [11] Sasu Tarkoma. 2012. *Publish/Subscribe Systems: Design and Principles*. John Wiley & Sons, Chichester, UK.